

Centro de Treinamento e Desenvolvimento da Indústria 4.0

Manual resumido sobre Realidade Aumentada e protocolo MQTT

Bruno Pinho Campos – Analista de Tecnologia

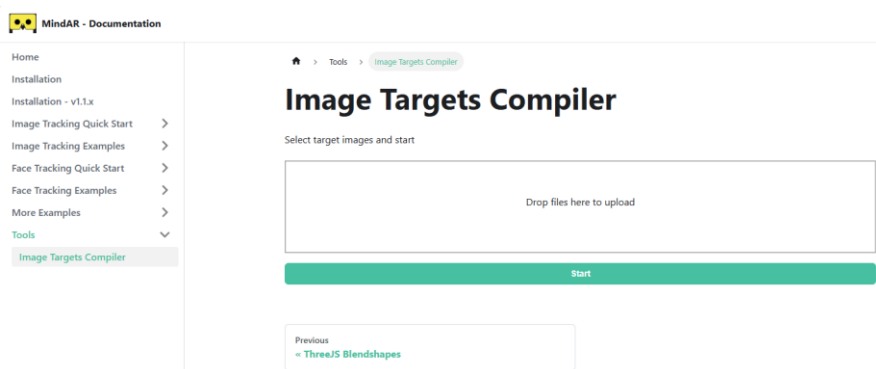
Criando aplicação RA

As tecnologias utilizadas para esta aplicação são HTML, CSS, Javascript, MindAR e A-Frame.

Criando o alvo usando MindAR

Documentação e Ferramentas MindAR: <https://hiukim.github.io/mind-ar-js-doc/>

Acesse a ferramenta “Image Targets Compiler” pelo link de documentação do MindAR, clicando no menu “Tools – Image Targets Compiler”.



Arraste sua imagem “jpg” do alvo, para a área retangular com a mensagem “Drop files here to upload”, em seguida clique no botão verde “Start” e aguarde o processamento.



Após o processamento, faça o download do arquivo gerado clicando no botão “Download compiled”. Será baixado um arquivo chamado “targets.mind”. Para facilitar podemos renomear o arquivo para “alvo.mind”.

Criando o código principal

```
01 <!DOCTYPE html>
02 <html lang="pt-BR">
03 <head>
04   <meta charset="UTF-8">
05   <meta name="viewport" content="width=device-width, initial-scale=1.0">
06   <title>Oficina de RA Industrial - SENAI MBI</title>
07
08   <script src="https://aframe.io/releases/1.4.2/aframe.min.js"></script>
09   <script src="https://cdn.jsdelivr.net/npm/mind-ar@1.2.2/dist/mindar-image-aframe.prod.js"></script>
10
11   <style>
12     body {
13       margin: 0;
14       overflow: hidden;
15     }
16   </style>
17 </head>
18
19 <body>
20   <a-scene mindar-image="imageTargetSrc: ./assets/alvo.mind;" color-space="sRGB"
21     renderer="colorManagement: true, physicallyCorrectLights: true, alpha: true" vr-mode-ui="enabled: false"
22     device-orientation-permission-ui="enabled: false">
23     <a-camera position="0 0 0" look-controls="enabled: false"></a-camera>
24     <a-entity mindar-image-target="targetIndex: 0">
25       <a-text id="textosimples" value="Alvo Encontrado!\nBem-vindo a AR" color="#00FF00" align="center"
26         position="0 0 0" scale="1 1 1"></a-text>
27     </a-entity>
28   </a-scene>
29 </body>
30 </html>
```

Explicação fundamental do código:

Linha 08: Importação do framework A-Frame

Linha 09: Importação da biblioteca MindAR

Linha 20: O coração da aplicação, tudo que for de RA irá acontecer dentro desta tag.

<a-scene>: É a tag que inicializa o "mundo" 3D na página web. Quando o navegador lê essa tag, ele cria um elemento **<canvas>**, configura a câmera, as luzes e prepara o motor de renderização (WebGL) em segundo plano.

mindar-image="imageTargetSrc: ./assets/alvo.mind;": Aqui é onde o **MindAR** entra em ação dentro do A-Frame.

- **mindar-image:** Diz à cena que esta será uma experiência de Realidade Aumentada baseada em rastreamento de imagens (Image Tracking).
- **imageTargetSrc: ./assets/alvo.mind;** É a configuração principal do MindAR. Ela aponta o caminho onde está o arquivo do alvo compilado (alvo.mind) que o motor de AR precisa procurar na câmera. O “./” indica que o arquivo do alvo está na mesma pasta que o index.html.

color-space="sRGB": O espaço de cor "sRGB" é o padrão usado pela maioria dos monitores e telas de celulares. Ao definir isso, garantimos que as cores dos objetos 3D ou textos sejam exibidas de forma consistente e com a cor correta, evitando que fiquem muito "lavadas" ou escuras.

renderer="colorManagement: true, physicallyCorrectLights": Este atributo configura como o motor gráfico vai desenhar os gráficos na tela:

- **colorManagement: true:** Trabalha em conjunto com o `color-space="sRGB"`. Ele avisa ao A-Frame para ajustar automaticamente as cores das texturas e materiais para que fiquem realistas na tela do usuário.
- **physicallyCorrectLights:false :** Muda a matemática de como a luz funciona no seu ambiente 3D para imitar o mundo real. A luz vai enfraquecer de forma realista conforme se distancia de um objeto. É essencial se você for colocar objetos 3D que esteja em `true`.

vr-mode-ui="enabled: false": Por padrão, o A-Frame foi criado para Realidade Virtual (VR) e costuma colocar um botãozinho escrito "Enter VR" ou um ícone de óculos no canto inferior direito da tela. Como estamos fazendo uma experiência de **Realidade Aumentada (AR)** para rodar direto na câmera do celular/webcam, esse botão não tem utilidade. O `"enabled: false"` simplesmente esconde esse botão para deixar a interface limpa.

device-orientation-permission-ui="enabled: false": Em dispositivos móveis (especialmente no iOS), os navegadores exigem que o usuário clique em um botão para permitir o acesso aos sensores de movimento do celular (giroscópio). O A-Frame, por padrão, mostra uma tela pedindo essa permissão.

- O MindAR já possui o seu próprio sistema interno de pedir permissão para acessar a câmera e iniciar a AR. Colocar `"enabled: false"` aqui impede que o A-Frame crie uma tela de permissão duplicada ou confusa, deixando que o MindAR controle todo o processo de inicialização.

Linha 22: Definição dos "olhos" do usuário dentro da aplicação.

<a-camera>: No A-Frame, tudo é uma "entidade" (um objeto no espaço 3D). A tag `<a-camera>` é uma entidade especial que atua como o ponto de vista do observador. Sem ela, o A-Frame até criaria a cena, mas você não teria um "ponto de vista" para enxergar nada.

position="0 0 0": Isso define onde a câmera está posicionada no espaço 3D usando os eixos **X, Y e Z**.

- **X (0):** Horizontal. Esquerda / Direita.
- **Y (0):** Vertical. Cima / Baixo.
- **Z (0):** Frente / Trás.

Ao colocar `0 0 0`, estamos cravando a câmera exatamente no centro (a origem) do mundo virtual.

No MindAR isso é importante porque Diferente de experiências 3D comuns ou de VR, onde a câmera costuma ficar em `"0 1.6 0"` (que simula a altura média dos olhos humanos a 1,60m do chão). Na Realidade Aumentada do MindAR, o seu celular é quem "dita" onde você está. Colocar a câmera no ponto `"0 0 0"` garante que a visão virtual esteja perfeitamente alinhada e sincronizada com a lente da câmera física do seu dispositivo, sem nenhum deslocamento artificial.

look-controls="enabled: false": Este é um ajuste de segurança crucial para que sua AR não "quebre". No A-Frame padrão, esse controle permite que o usuário olhe ao redor do mundo 3D clicando e arrastando com o mouse (no PC) ou deslizando o dedo pela tela (no celular).

- **Por que desativar (enabled: false)?** Na Realidade Aumentada baseada em imagem (como o nosso Alvo o "QR Code"), quem dita a rotação e a perspectiva das coisas é a posição do QR Code em relação à câmera física do seu celular.

Linha 23: Âncora entre o mundo real “QR Code físico” ao mundo virtual “Conteúdo mostrado na AR”.

<a-entity>: Como vimos anteriormente, no A-Frame, absolutamente tudo é construído com base em "entidades" (o A-Frame usa um padrão chamado Entity-Component-System).

- Uma <a-entity> por si só é um objeto vazio, invisível e sem função nenhuma. Ela é como uma caixa de papelão vazia.
- Ela só ganha vida e utilidade quando você adiciona **componentes** (atributos) nela. Ao adicionarmos os componentes do MindAR a essa entidade, transformamos essa "caixa vazia" no ponto central do nosso rastreamento de AR. Tudo o que você colocar *dentro* dessa entidade (como o <a-text> que você viu no código) será agrupado e se moverá junto com ela.

mindar-image-target: Este é o componente mágico criado pelo MindAR. É ele que transforma a nossa entidade vazia em um **alvo rastreável**.

- Quando você coloca isso na entidade, o MindAR entende que essa entidade deve ficar "invisível" ou "desativada" até que o motor de visão computacional encontre o alvo na câmera.
- Assim que o QR Code entra no campo de visão da câmera, o MindAR liga essa entidade e **cola as coordenadas dela (posição, rotação e escala) exatamente em cima do QR Code físico**. Se você inclinar o papel com o QR Code, essa entidade se inclina junto, levando o seu texto junto com ela.

targetIndex: 0: Lembra do arquivo “alvo.mind” que geramos no compilador do MindAR? Aquele arquivo não precisa conter apenas uma imagem; ele pode ser um pacote com várias imagens diferentes (você poderia compilar um QR Code, a capa de um livro e um cartão de visitas no mesmo arquivo).

- Na programação, as listas começam a contar a partir do número zero.
- Portanto, **targetIndex: 0** diz ao MindAR para ancorar essa entidade específica na **primeira imagem** que você fez upload lá no compilador.
- Se o seu arquivo “.mind” tivesse duas imagens diferentes e você quisesse mostrar coisas diferentes para cada uma delas, você criaria duas entidades no código: uma com targetIndex: 0 (para a primeira imagem) e outra <a-entity mindar-image-target="targetIndex: 1"> (para a segunda imagem).

Em resumo: essa linha diz ao sistema para ficar aguardando e, quando a imagem de índice zero (nosso QR Code) aparecer na câmera, colar essa entidade virtual invisível bem em cima dela, servindo de base para o conteúdo da aplicação que aparecerá.

Linha 20: Esta é a parte visível da nossa aplicação. Como esta tag está **dentro** da <a-entity> que acabamos de ver, tudo o que configurarmos aqui ficará preso e se moverá junto com o seu QR Code.

<a-text>: Esta é uma tag "primitiva" do A-Frame criada especificamente para renderizar textos 2D dentro de um ambiente 3D. O A-Frame cuida de gerar a geometria e a fonte padrão para que o texto seja legível em qualquer ângulo.

value="Alvo Encontrado!\nBem-vindo a AR.": Este é o conteúdo exato que será exibido na tela.

- O detalhe mais importante aqui é o \n. Na programação, esses dois caracteres juntos funcionam como a tecla "Enter". Eles dizem ao sistema para fazer uma **quebra de linha**.
-

color="#00FF00": Define a cor do texto usando o sistema de cores Hexadecimal (No caso a cor verde puro).

align="center": Este comando centraliza o texto. O A-Frame alinha o texto à esquerda por padrão.

position="0 0 0": Assim como na câmera, isso define a posição nos eixos **X (horizontal)**, **Y (vertical)** e **Z (profundidade)**. Porém, há um conceito vital aqui: **Posição Relativa**.

- Como o `<a-text>` é "filho" da entidade do alvo (o QR Code), a posição 0 0 0 não é o centro do universo 3D, mas sim o **centro exato do alvo "QR Code" físico**.
- O texto ficará colado exatamente na superfície de onde está o alvo.
- **Dica:** Se você quiser que o texto pareça estar "flutuando" alguns centímetros para frente (saindo do papel em direção ao usuário), você pode mudar o eixo Z (profundidade) para um valor positivo, como `position="0 0 0.5"`.

scale="1 1 1": Define o tamanho do objeto em relação ao seu tamanho original nos mesmos eixos X, Y e Z.

- O valor 1 1 1 significa 100% da escala padrão (nem ampliado, nem reduzido).
- Se o texto ficar muito pequeno quando você testar na câmera, você pode aumentá-lo alterando a escala, por exemplo: `scale="2 2 2"` (que dobra o tamanho do texto). Se ficar muito grande, pode usar `scale="0.5 0.5 0.5"` (metade do tamanho). Como é um texto plano em 2D, o último número (Z) não afeta muita coisa, mas é boa prática manter a proporção.

Rodando a aplicação

Opção 1: Usando npx http-server (A mais rápida e recomendada)

Se você já tem o Node.js e o NPM instalados, não precisa nem criar um arquivo de servidor ou instalar nada permanentemente. O NPM tem uma ferramenta chamada npx que executa pacotes sob demanda.

1. Abra o terminal na pasta onde está o seu `index.html` e o `alvo.mind`.
2. Digite o seguinte comando e aperte Enter:

```
npx -y http-server -p 8080 -a 0.0.0.0 -c-1
```

3. Se for a primeira vez que roda este código, ele vai perguntar se você deseja instalar o pacote `http-server`. Digite **y** para confirmar.
4. O terminal mostrará os endereços disponíveis (geralmente `http://127.0.0.1:8080`). Basta clicar ou copiar e colar no navegador.

Aqui está o que cada parte do comando faz:

- **npx:** Executa um pacote do Node.js sem precisar instalá-lo permanentemente no seu computador.
- **-y:** Aceita automaticamente a confirmação de download do pacote, pulando a pergunta "Need to

```
Starting up http-server, serving ./

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 3600 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
http://172.18.112.1:8080
http://172.25.10.62:8080
http://10.10.0.127:8080
http://127.0.0.1:8080
Hit CTRL-C to stop the server
```

install the following packages... OK to proceed? (y)".

- **http-server**: O nome do pacote que está sendo executado. Ele cria um servidor HTTP simples.
- **-p 8000**: Define a **porta** do servidor como **8000** (você acessará via `http://localhost:8000`).
- **-a 0.0.0.0**: Define o endereço IP de escuta. O valor **0.0.0.0** significa que o servidor estará disponível não apenas no seu computador (localhost), mas também para **qualquer outro dispositivo na mesma rede Wi-Fi/local** (como seu celular).
- **-c-1**: Desativa completamente o **cache** do navegador (-c de cache, -1 para desativar). Isso garante que qualquer alteração que você fizer nos seus arquivos apareça imediatamente ao atualizar a página, sem que o navegador salve versões antigas.

Opção 2: Criando um arquivo .bat para iniciar o servidor de forma simples

Crie um novo arquivo chamado "iniciar_servidor.bat". Neste arquivo digite os comandos a seguir.

```
@echo off
title Servidor Web - MBI 5G
echo INICIANDO SERVIDOR WEB (REDE LOCAL)
echo http://localhost:8000

npx -y http-server -p 8000 -a 0.0.0.0 -c-1
start http://localhost:8000

pause
```

Conexão com MQTT

Para realizarmos a comunicação MQTT iremos usar a biblioteca Paho, que é bem versátil. Paho suporta linguagens como Python, Java, C e JavaScript.

As urls a seguir contém documentação oficial do Paho:

- <https://eclipse.dev/paho/>
- <https://github.com/eclipse-paho/paho.mqtt.javascript>
- <https://www.hivemq.com/blog/mqtt-client-library-encyclopedia-paho-js/>

Código Paho para HTML e Javascript

Código 1: importar a biblioteca Paho no head do html.

```
<script src="https://cdn.jsdelivr.net/npm/paho-mqtt@1.0.1/mqttws31.min.js"></script>
```

Código 3: Principais variáveis/constantes.

```
const o_topico = document.getElementById("topico");
const o_pecasboas = document.getElementById("pecasboas");
const o_pecasruins = document.getElementById("pecasruins");
const o_pecastotais = document.getElementById("pecastotais");

const BROKER_HOST = 'mqtt-dashboard.com';
const BROKER_PORT = 8884; // Porta WebSocket SSL/WSS
const BROKER_USER = 'root';
const BROKER_PASS = 'admin';
const topico = "SENAI/MBI_09_2026";
const CLIENT_ID = "mbi_09_2026_" + Math.random().toString(16).substring(2, 8);

const client = new Paho.MQTT.Client(BROKER_HOST, BROKER_PORT, CLIENT_ID);
```

Código 2: Principais funções Javascript para conectar e receber mensagens de tópicos MQTT com o Paho. Este código deverá ser inserido dentro das tags `<script> código </script>`.

```
function onConnect() {
  console.log("Conectado ao broker MQTT com sucesso!");
  o_topico.setAttribute("value", topico);
  client.subscribe(topico);
  console.log(`Assinatura enviada para o tópico: ${topico}`);
}

function onFailure(responseObject) {
  console.log("Falha ao conectar: " + responseObject.errorMessage);
  console.log("Tentando nova conexão, em 5 segundos...");
  setTimeout(() => onConnect(), 5000);
}

function onConnectionLost(responseObject) {
  if (responseObject.errorCode !== 0) {
    console.log("Conexão perdida: " + responseObject.errorMessage);
  }
  console.log("Tentando nova conexão, em 5 segundos...");
  setTimeout(() => onConnect(), 5000);
}

function onMessageArrived(message) {
  try {
    console.log(message.payloadString)
    const jsonMsg = JSON.parse(message.payloadString);
    const payload = jsonMsg.payload ? jsonMsg.payload : jsonMsg;
    atualizarDados(payload);
  } catch (erro) {
    console.error("Erro ao processar JSON recebido:", erro);
  }
}

function atualizarDados(payload) {
  if (payload.robotModeData) {
    const r = payload.robotModeData;
    v_pecasboas = Number(r.pecasboas || 0);
    v_pecasruims = Number(r.pecasruims || 0);
    v_pecastotais = v_pecasboas + v_pecasruims;
    o_pecasboas.setAttribute("value", v_pecasboas);
    o_pecasruims.setAttribute("value", v_pecasruims);
    o_pecastotais.setAttribute("value", v_pecastotais);
  }
}
```

Código 3: Objeto com os parâmetros para conexão

```
const connectOptions = {
  useSSL: true,
  userName: BROKER_USER,
  password: BROKER_PASS,
  onSuccess: onConnect,
  onFailure: onFailure,
  timeout: 5
};
```

Código 4: Definição das funções do cliente e chamada de conexão

```
client.onConnectionLost = onConnectionLost;
client.onMessageArrived = onMessageArrived;
client.connect(connectOptions);
```

Snippets de códigos

Snippet 1: Associar um elemento de visualização HTML no script. Para isso, inserimos o parâmetro id na tag com o nome que desejamos, como no exemplo a seguir.

```
<a-plane position="0 0 0" rotation="0 0 0" width="2" height="1" color="#58A" id="plano"></a-plane>
```

No código Javascript criamos uma constante e usamos o método `getElementById` com o id do elemento que desejamos associar.

```
const plano = document.getElementById("plano");
```

Snippet 2: Criar uma função para gerir a atualização das informações na página. É interessante que esta função receba todos os dados que deverão ser atualizados. Nesta função, podemos usar o método `setAttribute` que receberá o atributo e o valor para o atributo.

```
const atualizarDados = (vpower, venabled, vrunning, vjoint0) => {
  power.setAttribute("value", `Ligado: ${vpower ? "Sim" : "Nao"}`);
}
```

Snippet 3: Ao receber uma mensagem de um tópico inscrito, a função **onMessageArrived** será acionada, esta função deverá receber a informação, prepara-la conforme o necessário e passar para a função **atualizarDados**.

```
function onMessageArrived(message) {
  try {
    const vpower = payload.robotModeData.robotPowerOn;
  } catch (erro) {
    console.error("Erro ao processar JSON. Mensagem recebida não era um JSON válido:", erro);
  }
}
```

Snippet 4: Exemplo de função para publicar mensagens em um tópico.

```
function publicarMensagem(topicoAlvo, valor) {
  const payload = valor.toString();
  const message = new Paho.MQTT.Message(payload);
  message.destinationName = topicoAlvo;
  message.retained = true; // Mude para false se não quiser reter
  message.qos = 0; // (Opcional) Qualidade de Serviço (QoS 0, 1 ou 2)
  if (client && client.isConnected()) {
    client.send(message);
    console.log(`[PUBLICADO] Tópico: ${topicoAlvo} | Valor: ${payload}`);
  } else {
    console.warn("Falha ao publicar: O cliente MQTT não está conectado.");
  }
}
```

Snippet 5: Adicionando um botão para interação. É importante ressaltar que esta área dos botões deve ser adicionado antes do `a-scene`.

```
<div class="areaBotoes">
  <button onclick="btnTeste()">teste</button>
</div>
```

É importante definir na classe de formação destes elementos, para que eles fiquem acima da área do A-Frame e do MindAR, configuramos `position` em `absolute`.

```
.areaBotoes{
```

```
position: absolute; /* Tira o elemento do fundo da página */
top: 20px;          /* Distância do topo da tela */
left: 20px;         /* Distância da esquerda */
z-index: 9999;      /* Força a ficar na FRENTE da câmera (A-Frame) */
pointer-events: auto; /* Garante que o navegador permita clicar nele */
}
```

Na área dos scripts crie a função que será chamada ao clicar no botão.

```
const btnTeste={()=>{
  console.log('clicado')
}}
```

Snippet 6: Muitas vezes é importante saber quando o alvo foi encontrado/visto e ou perdido. Isso é essencial para criar uma boa experiência de usuário (por exemplo: pausar o envio de comandos MQTT quando o robô não está na tela, ou mostrar uma mensagem "Aponte a câmera para o QR Code").

O MindAR facilita muito isso disparando dois eventos automáticos chamados **targetFound** (Alvo Encontrado) e **targetLost** (Alvo Perdido). Para capturar esses eventos, só precisamos fazer duas coisas simples: adicionar um id na entidade do seu alvo e usar o `addEventListener` no JavaScript.

Passo 1: Colocar um ID no seu Alvo

Na sua tag `<a-entity mindar-image-target...>`, adicione um id para que o JavaScript consiga encontrá-la:

```
<a-entity id="meu-alvo" mindar-image-target="targetIndex: 0">
  ...
</a-entity>
```

Passo 2: Adicionar os EventListeners no JavaScript

Lá no `<script>`, de preferência logo no início dele, você busca essa entidade e cria os "escutadores" de eventos:

```
const meuAlvo = document.getElementById("meu-alvo");

meuAlvo.addEventListener("targetFound", (event) => {
  console.log("ALVO ENCONTRADO! A AR está visível na tela.");
  // O que se deseja fazer
});

meuAlvo.addEventListener("targetLost", (event) => {
  console.log("ALVO PERDIDO! A AR sumiu da tela.");
  // O que se deseja fazer
});
```

Bons estudos.